

INNOMATİCS
RESEARCH LABS

Task 3: Python Coding & Problem-Solving

Objective

The objective of this task is to strengthen your **Python problem-solving, logical thinking, function implementation, and modular programming skills** using Visual Studio Code (VS Code).

You will solve **15 coding problems**, write reusable Python functions, import them into another `.py` file, and execute your programs using the VS Code terminal.

Submission Guidelines

1. Folder Structure

Create a folder named **`FullName_Task3_Python`** containing the following three files:

```
RahulSharma_Task3_Python/
│
├── solutions.py
├── main.py
└── problem_solving.ipynb
```

2. File Requirements

`solutions.py`

- Write all **15 solutions as separate functions**.
- Mention the question number above each function.
- Each function must **return the result**.

`main.py`

- Import all 15 functions from `solutions.py`.

- Call each function with the given input and at least one additional test case.
- Display the outputs using `print()`.
- Execute the file in the VS Code terminal using `python main.py`.

`problem_solving.ipynb`

- Write the **pseudocode for all 15 questions in your own words** using Markdown or comments.
- Write and execute the Python code for each question.
- Include the expected output and at least one additional test case.

3. Important Instructions

1. Complete all **15 questions** independently.
2. Understand the problem and write your logic before coding.
3. Do not copy solutions or hardcode outputs.
4. Keep your code organized with proper question numbers.
5. Ensure all three files are complete and working.

4. Final Submission

- Compress the entire folder into a **single ZIP file**.
- **File Name:** `Task3_Python.zip`
- Submit the ZIP file **strictly through the LMS**.

⚠️ **Important:** All three files are mandatory. Incomplete submissions will affect your evaluation.

Task submissions will be strictly evaluated for Internship Certification purposes.

Questions

Q1. Find the Frequency of Each Word in a Sentence

Write a Python program to count how many times each word appears in a given sentence and store the result in a dictionary.

Example Input:

python is easy and python is powerful

Expected Output:

{'python': 2, 'is': 2, 'easy': 1, 'and': 1, 'powerful': 1}

Q2. Find the Frequency of Each Character in a Word

Write a Python program to count how many times each character appears in a given word and store the result in a dictionary.

Example Input:

banana

Expected Output:

{'b': 1, 'a': 3, 'n': 2}

Q3. Find the First Non-Repeating Character in a Word

Write a Python program to find the first character that appears only once in a given word. If no such character exists, return `None`.

Example Input:

swiss

Expected Output:

w

Q4. Check Whether Two Strings Are Anagrams

Write a Python program to check whether two strings are anagrams.

Two strings are anagrams if they contain the same characters with the same frequencies, regardless of their order.

Example Input:

s1 = "listen"
s2 = "silent"

Expected Output:

True

Q5. Find the First Repeating Character in a String

Write a Python program to find the first character whose second occurrence is encountered while reading the string from left to right.

If no character repeats, return `None`.

Example Input:

programming

Expected Output:

r

Q6. Move All Zeros to the End of a List

Write a Python program to move all zeros to the end of a list while maintaining the original order of non-zero elements.

Example Input:

[0, 3, 0, 1, 2, 0, 4]

Expected Output:

[3, 1, 2, 4, 0, 0, 0]

Q7. Find the Second Highest Distinct Number in a List

Write a Python program to find the second-largest distinct number in a given list.

Duplicate values should not be considered separate ranks.

If a second distinct number does not exist, return `None`.

Example Input:

[10, 20, 40, 30, 40, 20]

Expected Output:

30

Q8. Remove Duplicates Without Changing the Original Order

Write a Python program to remove duplicate elements from a list while preserving the order of their first occurrence.

Example Input:

[4, 2, 4, 5, 2, 7, 5]

Expected Output:

[4, 2, 5, 7]

Q9. Find the Missing Number from 1 to N

Write a Python program to find the missing number in a list containing numbers from `1` to `N`.

The list contains exactly one missing number, and all remaining numbers appear only once.

Example Input:

nums = [1, 2, 4, 5, 6]
N = 6

Expected Output:

3

Q10. Find Two Numbers Whose Sum Equals a Target

Write a Python program to find the first pair of numbers whose sum equals a given target.

Return the pair as a list. If no valid pair exists, return an empty list.

Return the pair whose second element is encountered earliest when scanning the input from left to right. If multiple earlier elements qualify, choose the earliest one.

Example Input:

nums = [2, 7, 11, 15]
target = 9

Expected Output:

[2, 7]

Q11. Find the Number Closest to Zero

Write a Python program to find the number closest to zero in a given list.

If two numbers are equally close to zero, return the positive number.

Example Input:

[-8, -3, 2, 5, -2]

Expected Output:

2

Q12. Find Common Elements Between Two Lists

Write a Python program to find the common elements between two lists.

Each common element should appear only once in the result, maintaining the order of its first occurrence in the first list.

Example Input:

list1 = [1, 2, 3, 4, 5]
list2 = [3, 4, 5, 6, 7]

Expected Output:

[3, 4, 5]

Q13. Rotate a List to the Right by K Positions

Write a Python program to rotate the elements of a list to the right by K positions.

Elements shifted beyond the last position should move to the beginning of the list.

Example Input:

nums = [1, 2, 3, 4, 5]
K = 2

Expected Output:

[4, 5, 1, 2, 3]

Q14. Find the Maximum Sum of Two Consecutive Elements

Write a Python program to find the maximum sum of any two consecutive elements in a list.

Assume the list contains at least two elements.

Example Input:

[4, 7, 2, 9, 5]

Expected Output:

14

Q15. Find All Pairs of Numbers Whose Sum Equals a Target

Write a Python program to find all unique pairs of numbers whose sum equals a given target.

Each pair should contain two distinct list positions, and duplicate pairs should not be included.

Return each pair in ascending numerical order. Return the list of pairs sorted by their first element. If no valid pairs exist, return an empty list.

Example Input:

nums = [2, 4, 3, 5, 7, 8, 1]
target = 9

Expected Output:

[[1, 8], [2, 7], [4, 5]]